

standard_backend_competency_framework

Backend Competency Framework v1

Introduction

This framework defines the core competencies expected of Backend Developers at the Junior level, and serves as the reference standard against which curriculum, assessment items, and performance reviews should be aligned. Any curriculum module, assessment item, or performance evaluation criterion that cannot be traced back to a specific competency listed here should be treated as out of scope until the framework itself is formally revised to include it.

This framework is deliberately scoped to Junior-level expectations. A separate framework, not included in this document, defines expectations for Mid-level and Senior Backend Developers, building on these same competency domains at greater depth and breadth.

Competency Domain 1: Programming Fundamentals

1.1 Language Proficiency

Ability to write correct, idiomatic code in the team's primary backend language, following established style conventions without requiring constant correction during code review.

1.2 Data Structures and Algorithms

Ability to select and apply appropriate data structures for common backend problems — such as choosing a hash map for deduplication or a queue for task processing — without defaulting to a single familiar structure regardless of fit.

1.3 Error Handling

Ability to anticipate realistic failure modes (invalid input, unavailable dependencies, unexpected data shapes) and handle them gracefully with clear, actionable errors, rather than allowing unhandled exceptions to propagate to the caller or, worse, to the end user.

Competency Domain 2: Data Management

2.1 Relational Database Design

Ability to design normalized schemas appropriate to the application's actual access patterns, making deliberate, explainable tradeoffs rather than either duplicating data carelessly or over-normalizing to the point of requiring excessive joins for simple queries.

2.2 Query Writing and Optimization

Ability to write correct SQL for common data retrieval and modification needs, and to identify obviously inefficient queries (such as those causing a full table scan on a large table) using basic query plan analysis.

2.3 Data Migration Safety

Awareness that schema migrations affecting a live production system require backward compatibility considerations — such as staging a new required column as nullable first, backfilling data, and only then enforcing a not-null constraint — rather than applying a single large migration that could lock a table or break currently-running application code expecting the old schema.

Competency Domain 3: API and Service Design

3.1 RESTful API Design

Ability to design resource-oriented APIs following consistent conventions for naming, HTTP method usage, and status codes, rather than mixing action-based and resource-based endpoint styles within the same API surface.

3.2 Service Boundaries

Basic understanding of when and why functionality should be separated into distinct services versus kept within a single service, including an awareness of the added operational complexity that comes with splitting a system into multiple services.

3.3 API Documentation

Ability to produce and maintain API documentation that remains accurate as the implementation evolves, ideally through tooling that generates documentation from the implementation or its specification rather than a manually maintained document prone to drifting out of sync.

Competency Domain 4: Quality and Testing

4.1 Unit Testing

Ability to write isolated, meaningful unit tests for new code, covering both expected-path and edge-case scenarios, without relying on manual verification as the primary means of confirming correctness.

4.2 Debugging

Ability to systematically diagnose and resolve defects using appropriate tooling (logs, debuggers, or tracing) rather than speculative, unstructured trial and error.

4.3 Code Review Participation

Ability to give constructive, specific, and actionable code review feedback to peers, and to receive and engage productively with feedback on one's own pull requests.

Competency Domain 5: Security and Operations

5.1 Security Fundamentals

Awareness of common vulnerability classes (SQL injection, cross-site scripting, cross-site request forgery) and the standard mitigations for each, applied consistently in day-to-day code rather than treated as separate “security work” handled by someone else.

5.2 Secrets and Credential Handling

Consistent avoidance of hardcoded secrets and credentials in source control, and a clear understanding that a secret exposed in git history must be rotated, not merely removed from the current file state.

5.3 Basic Observability

Ability to add appropriate logging, with sufficient context and correctly chosen severity levels, to diagnose issues in a running system without needing to reproduce the issue locally from scratch every time.

Using This Framework

Curriculum designers should map every course or training module to at least one competency listed here, explicitly noted in the module's outline. Assessment item writers should map every individual item to exactly one sub-competency (for example, "2.1 Relational Database Design"), not a broad domain, to keep evaluation results interpretable at a useful level of granularity. Gaps identified during a competency-gap analysis should reference this framework's specific numbering for full traceability from gap, to module, to assessment item, to evaluation result.

Revision Notes

This is Version 1 of the framework. Future revisions should be tracked with an explicit changelog documenting what changed and why, rather than silently overwriting the previous version, so that historical curricula and assessments remain traceable to the specific framework version they were designed and validated against — an assessment item validated against Version 1's definition of a competency may no longer be valid evidence of that competency if Version 2 redefines it.